

RBF-GNN: RATIONAL BASIS FUNCTIONS FOR PSEUDO-COORDINATE BASED GRAPH CONVOLUTIONS

Anonymous authors

Paper under double-blind review

ABSTRACT

We propose RBF-GNN, a new pseudo-coordinate based graph neural network architecture that takes into account euclidean, spherical or angular coordinates and uses them to induce a powerful spatial inductive bias. Similar in architecture to SplineCNN, we improve upon the latter by replacing the less efficient sparse-activation based B-splines whose number grows exponentially with dimension by rational Padé basis functions. For effective training we propose a spline-subspace initialization and a variance-preserving weight rescaling. Experimentally, we evaluate on a number of popular neural network architectures that use SplineCNNs. We replace only the SplineCNNs with RBF-GNN. We achieve improved results, including on semantic keypoint matching, shape matching, event based camera computer vision tasks. We will make our implementation publicly available upon acceptance of the paper.

1 INTRODUCTION

Graph neural networks (GNN) encode geometric relationships between points. For graphs embedded in some coordinate space, for example Euclidean or spherical ones, SplineCNNs Fey et al. (2018) are one of the best-performing methods that make explicit use of coordinates and enable higher performance than coordinate-free GNNs. In particular, SplineCNN’s coordinate-based convolution allows for efficient spatial localization and encodes a powerful inductive spatial bias. Since many geometric tasks have relatively little data, this inductive bias is especially helpful. This has resulted in SplineCNN being used, to our knowledge, in all state of the art semantic keypoint matching methods ? for geometry based feature refinement.

SplineCNN works by modulating convolutions based on the activations of spatially localized B-splines. However, SplineCNN has a few drawbacks:

Sparsity: B-splines form a partition of unity and for every coordinate only a few of them become non-zero. Hence, most of the convolutions are not active. This in turn makes it difficult to become computationally efficient on GPUs and means that convolutions receive relatively sparse updates during backpropagation.

Efficiency: In order to localize activations spatially with enough precision in high dimensions, a fine enough partition of unity must be employed, necessitating a large enough B-spline bases. The size of this B-splines basis grows exponentially with the dimension of the coordinate space.

We propose to improve SplineCNN’s architecture by replacing splines by rational safe Padé basis functions. Their advantages are:

- more flexible, learnable localization
- increase computational efficiency and decrease required parameter count since fewer activation functions are needed due to them not being zero at most places and
- better scaling to higher dimensions, since the size of a B-spline basis grows exponentially with the number of coordinate dimensions.

We also show empirically the advantages of RBF-GNN. We outperform SplineCNN based architectures by swapping SplineCNN for RBF-GNN and leaving everything else the same on the following benchmarks:

- Sparse keypoint matching on PascalVOC and SPair-71K with the Deep Graph Matching Consensus architecture as well as the more recent ...
- Shape matching on FAUST.
- Event-based camera computer vision.

2 RELATED WORK

Geometry-aware graph convolutions. Message-passing neural networks provide the general neighbourhood-aggregation view (Gilmer et al., 2017), while spatial operators additionally condition messages on edge geometry. ECC generates filters from edge attributes (Simonovsky & Komodakis, 2017); MoNet applies Gaussian mixtures to pseudo-coordinates (Monti et al., 2017); and FeaStNet learns feature-dependent assignments to weight matrices (Verma et al., 2018). Continuous filters have also been generated by neural networks for molecules (Schütt et al., 2017), point clouds (Wang et al., 2018; Wu et al., 2019), and general graphs equipped with positional encodings (Ma et al., 2024). Related point-cloud operators use Taylor-polynomial filters (Xu et al., 2018), learnable kernel points (Thomas et al., 2019), geometry-conditioned weight banks (Xu et al., 2021), or relative-position attention (Zhao et al., 2021). SplineCNN instead expands a continuous kernel in compactly supported tensor-product B-splines (Fey et al., 2018), an operator subsequently used for graph matching (Fey et al., 2020) and event streams (Schaefer et al., 2022). RBF-GNN is closest to SplineCNN: it preserves its message-passing rule and matrix bank, but replaces the fixed grid basis by learnable non-separable rational functions whose number is independent of k^D .

Learned continuous and rational bases. Learnable functional parameterisations include spline time-frequency atoms (Balestrierio et al., 2018), neural continuous kernels (Romero et al., 2022b;a), and coordinate networks based on Fourier features, periodic activations, or multiplicative filters (Tancik et al., 2020; Sitzmann et al., 2020; Sahu et al., 2021). Rational functions have been studied as trainable activations (Molina et al., 2020) and as efficient smooth approximators (Boullé et al., 2020). More recently, KAN places learned B-spline functions on network edges (Liu et al., 2025), while KAT replaces those splines by rational functions to improve efficiency and scalability (Yang & Wang, 2025). Rational-activation and KAN-family models modify neuron or feature-mixing nonlinearities, while neural coordinate models learn an entire continuous signal or kernel. In contrast, our rational functions are multivariate mixing coefficients of a matrix-valued graph kernel and are initialised to preserve both the SplineCNN kernel subspace and its output variance.

3 METHOD

RBF-GNN is a graph convolution whose continuous kernel is spanned by learnable rational basis functions of pseudo-coordinates. We first set up pseudo-coordinates and the continuous-kernel convolution operator, then introduce the rational basis in safe Padé form. A longer remark relates the construction to SplineCNN (Fey et al., 2018), which uses the same operator with a fixed B-spline basis and for which RBF-GNN is a drop-in replacement. Finally, we assemble the full network and describe the initialization that lets it train as reliably as the spline model it can replace.

Graphs and pseudo-coordinates. We consider graphs $G = (V, E)$ embedded in a coordinate space: every node i carries features $\mathbf{x}_i \in \mathbb{R}^{C_{\text{in}}}$ and a spatial position $\mathbf{z}_i \in \mathbb{R}^D$ (2-D image keypoints, vertices of a 3-D mesh, ...). The convolution below depends on *relative* positions. Because the kernel will be parameterised by basis functions living on a *fixed, bounded* domain, the offsets are affinely rescaled into the unit cube: with δ_{max} the largest absolute offset component occurring in the graph, the *pseudo-coordinate* of edge (j, i) is

$$\mathbf{u}_{ij} = \frac{\mathbf{z}_j - \mathbf{z}_i}{2\delta_{\text{max}}} + \frac{1}{2}\mathbf{1} \in [0, 1]^D \quad (1)$$

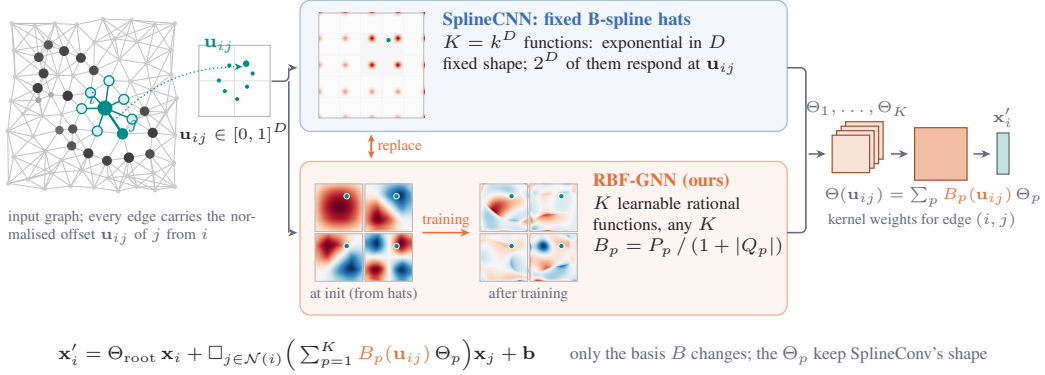


Figure 1: **RBF-GNN in one picture.** Every edge of the input graph carries a pseudo-coordinate $\mathbf{u}_{ij} \in [0, 1]^D$, the normalised offset of j from i . A continuous-kernel layer turns it into kernel weights through a basis B , and this basis is the only thing we change: SplineCNN uses k^D fixed B-spline hats, RBF-GNN uses a small, freely chosen number of learnable rational functions, initialised in the spline subspace and reshaped by training (shown: the functions of one layer before and after training on MNIST superpixels).

The zero offset lands at the centre $\frac{1}{2}\mathbf{1}$, a neighbour to the left/below has $u_d < \frac{1}{2}$ in that dimension, and the largest offset touches the boundary. See Figure 2 for an illustration of the pseudo-coordinate construction. These pseudo-coordinates are the only way geometry enters the model. Polar or spherical encodings can be handled analogously.

Continuous convolution kernels. An image convolution assigns a separate weight matrix to each of the finitely many pixel offsets. On an irregular graph the offsets $\mathbf{u}_{ij}$ vary continuously, so we instead learn a *kernel function* $g: [0, 1]^D \rightarrow \mathbb{R}^{C_{\text{in}} \times C_{\text{out}}}$ and evaluate it at the observed pseudo-coordinates. The kernel is parameterised as a linear combination $g(\mathbf{u}) = \sum_{p=1}^K B_p(\mathbf{u}) \Theta_p$ of K scalar basis functions $B_p: [0, 1]^D \rightarrow \mathbb{R}$ with learnable weight matrices $\Theta_p \in \mathbb{R}^{C_{\text{in}} \times C_{\text{out}}}$, and a layer computes

$$\mathbf{x}'_i = \Theta_{\text{root}} \mathbf{x}_i + \square_{j \in \mathcal{N}(i)} \left(\sum_{p=1}^K B_p(\mathbf{u}_{ij}) \Theta_p \right) \mathbf{x}_j + \mathbf{b}, \quad (2)$$

with a separate root weight for the node itself, a bias, and aggregation $\square \in \{\text{mean}, \text{add}\}$ over the neighbourhood. This operator was introduced by SplineCNN (Fey et al., 2018); the difference lies entirely in the choice of the basis functions B_p , which are fixed B-splines there (see the remark below) and learnable rational functions here.

Rational safe Padé basis functions. In RBF-GNN the scalar basis functions $B_p: [0, 1]^D \rightarrow \mathbb{R}$ of (2) are learnable rational functions. Each B_p is a quotient of two polynomials whose coefficients are network parameters. We build up the functional form in one dimension first. A *polynomial* of degree m with learnable coefficients $a_0, \dots, a_m$ is

$$P(t) = \sum_{j=0}^m a_j t^j. \quad (3)$$

A *rational* function of degrees (m, n) is a quotient of two such polynomials,

$$R(t) = \frac{P(t)}{Q(t)} = \frac{\sum_{j=0}^m a_j t^j}{\sum_{j=0}^n b_j t^j}. \quad (4)$$

Training a free denominator is unstable, since it can become zero, making the evaluation undefined, but also being close to zero will lead to numerical overflow. We therefore use the *safe* Padé form (Molina et al., 2020), in which the denominator polynomial Q (without constant term) enters only through

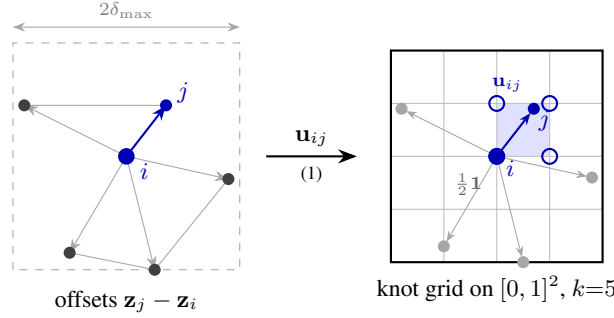


Figure 2: Pseudo-coordinates ($D=2$). **Left:** the convolution at node i sees each neighbour j only through its spatial offset $\mathbf{z}_j - \mathbf{z}_i$ (arrows); the dashed box marks the largest offset magnitude $\delta_{\max}$ occurring in the graph. **Right:** the shift-and-scale map (1) sends the dashed box to the unit square, so the neighbourhood on the left reappears as a scaled copy: node i (zero offset) lands at the centre $\frac{1}{2}\mathbf{1}$, each neighbour at its pseudo-coordinate — j at $\mathbf{u}_{ij}$ (blue) — and the largest offset touches the boundary (bottom dot). The B-spline kernel is defined once on this square via the $k \times k$ knot grid: at $\mathbf{u}_{ij}$ only the $2^D=4$ hats anchored at the circled corners of the containing cell (shaded) are non-zero, so only 4 of the $K=25$ weight matrices $\Theta_{\mathbf{p}}$ are used — and updated — for this edge.

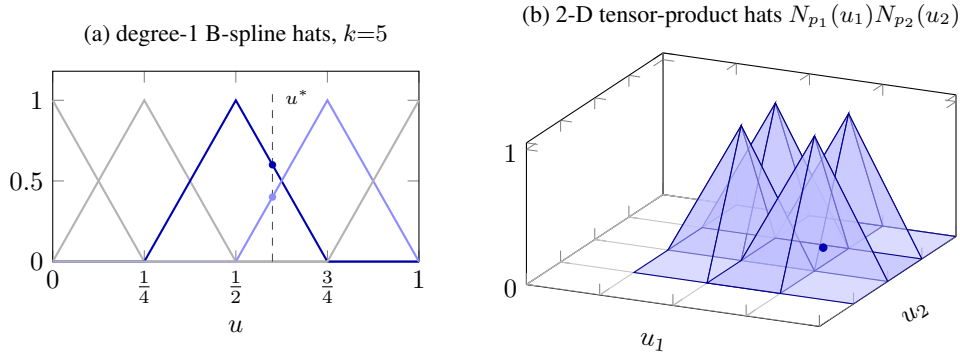


Figure 3: The B-spline basis. **(a)** The five open degree-1 B-splines on $[0, 1]$; at any coordinate u^* (dashed) only two are non-zero (dots), so in D dimensions only 2^D of the k^D tensor products activate. **(b)** 3-D view of the 2-D basis: each $B_{\mathbf{p}}$ is a pyramid $N_{p_1}(u_1)N_{p_2}(u_2)$ centred on a knot pair. Shown are the four pyramids anchored at the corners of the shaded cell of Figure 2; they are the only ones of the $K = k^2 = 25$ basis functions that are non-zero at the pseudo-coordinate $\mathbf{u}_{ij}$ (dot).

$$1 + |Q(\cdot)| \geq 1:$$

$$B(t) = \frac{P(t)}{1 + |Q(t)|} = \frac{\sum_{j=0}^m a_j t^j}{1 + \left| \sum_{j=1}^n b_j t^j \right|}, \quad (5)$$

which remains well defined and bounded for every value of the learnt coefficients a_j, b_j . Figure 4 contrasts the plain rational (4) with the safe form (5).

In practice we expand P and Q not in the monomial basis $1, t, t^2, \dots$ but in Chebyshev polynomials of the first kind $\phi_0, \phi_1, \dots$ ($\phi_0=1, \phi_1=t, \phi_{j+1}=2t\phi_j - \phi_{j-1}$), i.e. $P(t) = \sum_j a_j \phi_j(t)$. This parametrization spans the same function class but is better conditioned on bounded domains. the map from monomial coefficients to polynomial values on an interval is exponentially ill-conditioned in the degree (?), so with monomials a small step in a high-order coefficient barely moves the function near $t = 0$ but changes it violently near the boundary, giving gradients on wildly different scales. The Chebyshev polynomials, by contrast, are orthogonal and uniformly bounded, $|\phi_j(t)| \leq 1$ on $[-1, 1]$, so every coefficient acts on a comparable scale and $|a_j|$ directly bounds the contribution of the j -th term; Chebyshev expansions are the standard well-conditioned representation of polynomials on bounded intervals in numerical practice (?).

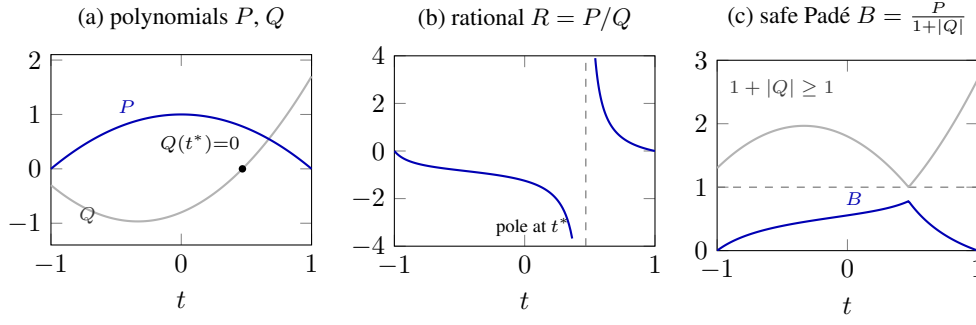


Figure 4: From polynomials to safe Padé basis functions, Eqs. (3)–(5), with the same numerator P and denominator Q in all three panels. **(a)** Two polynomials with learnable coefficients; Q has a root t^* inside the domain. **(b)** The plain rational function $R = P/Q$ diverges at t^* , and any root of Q can move into the domain during training. **(c)** In the safe form the denominator $1 + |Q|$ (gray) never drops below one (dashed), so B stays bounded and well defined for every setting of the coefficients; where Q vanishes, B simply equals P .

Below we extend this one-dimensional basis function to multiple dimensions in two different ways.

Product rational basis. Each of the K basis functions factorises over the coordinate dimensions into univariate safe Padé functions (5),

$$B_{\mathbf{p}}(\mathbf{u}) = \prod_{d=1}^D \frac{P_{d,p}(u_d)}{1 + |Q_{d,p}(u_d)|}, \quad (6)$$

where every factor has its own learnable polynomials $P_{d,p}$ and $Q_{d,p}$.

Multivariate rational basis. Each basis function $B_p: [0, 1]^D \rightarrow \mathbb{R}$ is a single non-separable rational function of all coordinates jointly:

$$B_p(\mathbf{u}) = \frac{P_p(\mathbf{u})}{1 + |Q_p(\mathbf{u})|}, \quad P_p(\mathbf{u}) = \sum_{|\alpha| \leq m} a_{p,\alpha} u_1^{\alpha_1} \cdots u_D^{\alpha_D}, \quad Q_p(\mathbf{u}) = \sum_{1 \leq |\alpha| \leq n} b_{p,\alpha} u_1^{\alpha_1} \cdots u_D^{\alpha_D}, \quad (7)$$

with multi-indices $\alpha \in \mathbb{N}_0^D$ of total degree $|\alpha| \leq m$ and $|\alpha| \leq n$. As in the univariate case, the implementation uses the Chebyshev parametrization. Each monomial $u_1^{\alpha_1} \cdots u_D^{\alpha_D}$ is replaced by the product $\phi_{\alpha_1}(t_1) \cdots \phi_{\alpha_D}(t_D)$, spanning the same function class in better-conditioned form.

Original SplineCNN and B-spline basis. The original SplineCNN Fey et al. (2018) uses the operator (2), but chooses the basis functions B_p as B-splines, see Figure 3. This construction yields spatial localisation and a strong geometric inductive bias. However, in every coordinate at most 2 B-splines are active, which means in larger dimensions that all but 2^D of the $\Theta_{\mathbf{p}}$ receive zero gradient. This results in sparse, uneven updates and dense evaluation wasted on zeros. The basis size $K = k^D$ grows exponentially with the coordinate dimension D .

Initialization. Randomly initializing the basis functions works consistently worse. We observe -0.7 to -5.8 drops in accuracy, see the ablation in our experiments. Since we use small random numbers for initialization the resulting basis functions are approximately flat. First, spatial localization would need to be completely learned from scratch. Second, near-identical basis activations give the different Θ_p strongly correlated gradients. Therefore, we propose two initializations that allow for spatial localization from the beginning.

Spline fit. Fit each rational function to its B-spline by least squares so the layer starts as approximately SplineConv. We perform a stage-wise optimization to fit our learnable basis functions as follows:

1. We fit the numerator P with a closed-form least squares fit.

2. Next, we refine both P and Q with Adam (lr 0.01 for 500 steps).
3. Last, for further numerical accuracy we use L-BFGS for 100 iterations on float64.

We evaluate squared loss on a fixed grid of points, using 256 for 1D, 32^2 for 2D and 16^2 on 3D. We use a small values for the denominator Q since the gradient of a constant zero Q would vanish. The fit is cached, so stacked layers share the same fit. The spline fit can be performed whenever the number of basis functions $K = k^d$ for some $k \in \mathbb{N}$.

PCA of splines. When $K \neq k^D$, we cannot fit to the splines. Instead, we fit the K functions to the K leading principal components of the k^D spline basis. In detail, this is done as follows: We sample points on the same grid as above. We collect the values in a matrix A , compute its SVD and the the K leading left singular vectors $u_1, \dots, u_K$. We rescale each u_i to unit maximum and fix to a deterministic sign. Last, we fit each u_i with the same procedure as above for the spline fit.

Variance-preserving gain. The initialization of the kernel weights is calibrated for the spline basis. The entries of Θ_p are drawn i.i.d. zero-mean with variance proportional to $1/(KC_{\text{in}})$ (§??). Thus, the message $\sum_p B_p(\mathbf{u}) \Theta_p \mathbf{x}_j$ along an edge has variance proportional to the basis energy $\sum_p B_p(\mathbf{u})^2$. For partition-of-unity basis like the spline the variance remains. However, the PCA-based initialization can result in larger activations and, without correction, to inflated output scale.

Adapting a technique from KAT (Yang & Wang, 2025), we therefore rescale the kernel weights by $\sqrt{\alpha}$ after basis initialization. The gain compares the energy of the B-spline basis being replaced with that of the initialized rational basis,

$$\alpha = \mathbb{E}_{\mathbf{u}} \left[\sum_{\mathbf{p}} N_{\mathbf{p}}(\mathbf{u})^2 \right] / \mathbb{E}_{\mathbf{u}} \left[\sum_p B_p(\mathbf{u})^2 \right], \quad (8)$$

where $N_{\mathbf{p}}$ are the k^D tensor-product B-spline hats of Figure 3, B_p are our K rational basis functions as returned by the spline or PCA fit above, and both expectations are over uniform $\mathbf{u}$, evaluated on the fit grid. The initial message variance then matches that of SplineConv with the same K .

Efficient implementation. A naive implementation of RBF-GNN materialises the Chebyshev feature tensor before contracting it with the coefficients. More efficiently, we provide a triton kernel that never materializes this intermediate tensor that directly produces only K basis values from the D pseudo-coordinates. The polynomial sums are efficiently computed by a Horner-like scheme. The resulting kernel runs about $15x$ times faster than the naive PyTorch implementation and $3x$ faster than a compiled variant and consumes less memory. Training and inference with the Triton kernel for RBF-GNN is slightly faster overall than the PyG-implementation of SplineCNN.

4 EXPERIMENTS

Methods

SplineCNN:

MLP:

RBF-GNN-uv:

RBF-GNN-mv:

Keypoint matching on PascalVOC and SPair-71k. We evaluate semantic keypoint matching in two host pipelines that both use a continuous-kernel graph convolution for geometric feature refinement: DGM (Fey et al., 2020) and NMT. In each pipeline we replace only the SplineConv layers and leave the rest of the architecture, the backbone and the training recipe untouched. Table 1 reports every configuration in both pipelines on PascalVOC-Keypoints and SPair-71k.

Method	K	DGMC (Fey et al., 2020)			NMT		
		Params ↓	PascalVOC ↑	SPair-71k ↑	Params ↓	PascalVOC ↑	SPair-71k ↑
<i>Spline baselines</i>							
SplineCNN	25	9.50M	73.00 ± 0.40	76.18 ± 0.64	28.17M	—	82.10 ± 0.11
SplineCNN	9	3.74M	72.78 ± 0.40	76.04 ± 0.43	10.84M	—	81.54 ± 0.31
SplineCNN	4	1.93M	71.48 ± 0.40	71.92 ± 0.38	5.42M	—	77.79 ± 0.40
<i>MLP learned-basis control</i>							
MLP basis	4	1.94M	72.76 ± 0.27	73.74 ± 0.71	5.42M	—	80.40 ± 0.57
MLP basis	9	3.74M	72.04 ± 0.73	74.34 ± 0.62	10.84M	—	81.05 ± 0.32
<i>RBF-GNN with a product basis</i>							
RBF-GNN (product)	25	9.50M	72.88 ± 0.48	76.34 ± 0.42	28.17M	—	82.08 ± 0.21
RBF-GNN (product)	9	3.74M	73.26 ± 0.18	76.24 ± 0.73	10.84M	—	81.64 ± 0.35
<i>RBF-GNN with a multivariate basis</i>							
RBF-GNN (mv), hat init	9	3.74M	74.12 ± 0.48	77.68 ± 0.35	10.84M	—	82.11 ± 0.35
RBF-GNN (mv), PCA+vp	4	1.94M	73.78 ± 0.41	77.44 ± 0.36	5.42M	—	81.52 ± 0.22
RBF-GNN (mv), PCA+vp	6	2.66M	74.06 ± 0.37	77.80 ± 0.66	7.59M	—	81.71 ± 0.18
RBF-GNN (mv), PCA+vp	9	3.74M	73.72 ± 0.19	77.50 ± 0.27	10.84M	—	81.59 ± 0.14
RBF-GNN (mv), PCA+vp	16	6.26M	73.26 ± 0.42	77.14 ± 0.40	18.42M	—	81.69 ± 0.27
RBF-GNN (mv), PCA+vp	2	1.21M	71.76 ± 0.45	—	—	—	—
RBF-GNN (mv), PCA+vp	1	0.85M	68.26 ± 0.36	—	—	—	—
<i>RBF-GNN degree control</i>							
RBF-GNN (mv (5, 4))	25	9.51M	73.08 ± 0.25	76.26 ± 0.23	28.17M	—	82.01 ± 0.21

Table 1: Semantic keypoint matching in two host pipelines. Accuracy (%) is mean ± std over 5 seeds; **best**, **second**, and **third** mark the three highest means *within each benchmark column*. K is the number of basis functions and is shared by both pipelines; the DGMC parameter counts are whole-model totals, whereas the NMT counts cover the graph-convolution stack only and exclude the frozen VGG16 backbone, so the two parameter columns are not comparable to each other. Metrics follow each pipeline’s own protocol and are likewise not comparable across columns: DGMC/PascalVOC is averaged over the 20 object categories, DGMC/SPair-71k is pair-averaged over 18 categories, and NMT/SPair-71k is keypoint accuracy over 18 classes. Only SPair-71k under DGMC has a validation split, so its column reports the validation-selected checkpoint; the remaining columns report the best test epoch. Entries marked “—” were not run: NMT has no PascalVOC runs, and $K \in \{1, 2\}$ was swept on DGMC/PascalVOC only.

The multivariate rational basis is the strongest configuration in all three populated columns. Its margin over the best spline baseline shrinks as the host pipeline strengthens — +1.12 on DGMC/PascalVOC, +1.62 on DGMC/SPair-71k and +0.01 on NMT/SPair-71k — so the accuracy gain is pipeline-dependent. The margin over the MLP learned-basis control does not follow that pattern (+1.36, +3.46 and +1.06), which indicates that the structured rational form matters beyond merely learning the basis. The parameter-efficiency result is the more robust one: a multivariate basis matches or exceeds the widest spline grid in every column while using 2.6–4.9× fewer parameters. At matched K the advantage is concentrated at small bases (+0.6 to +5.5 for $K \in \{4, 9\}$) and vanishes at $K=25$.

Shape correspondence on FAUST. We evaluate dense shape correspondence on FAUST following the SplineCNN protocol (Fey et al., 2018). The task assigns every vertex of a held-out human mesh to a template vertex, and we report exact-match accuracy over three random seeds.

Method	K	Params ↓	Final ↑
<i>Spline baselines</i>			
<i>SplineCNN $k=5$ (published)</i>	125	4.10M	99.20
SplineCNN $k=5$, mean + clip	125	4.11M	98.71 $\pm$ 0.08
SplineCNN $k=5$, add + clip	125	4.11M	97.23 $\pm$ 0.51
SplineCNN $k=5$, add, no clip	125	4.11M	29.1 $\pm$ 50.3
SplineCNN $k=3$	27	2.30M	83.6 $\pm$ 15.6
SplineCNN $k=2$	8	1.95M	3.4 $\pm$ 3.3
<i>MLP learned-basis control</i>			
MLP basis, $K=8$	8	1.96M	98.21 $\pm$ 1.11
<i>RBF-GNN with a product basis</i>			
RBF-GNN (product), $k=3$	27	2.30M	99.24 $\pm$ 0.08
<i>RBF-GNN with a multivariate basis</i>			
RBF-GNN (multivariate), $K=4$	4	1.89M	99.38 $\pm$ 0.09
RBF-GNN (multivariate), $K=4$, mean aggr.	4	1.89M	98.93 $\pm$ 0.10
RBF-GNN (multivariate), $K=8$	8	1.97M	99.41 $\pm$ 0.04
RBF-GNN (multivariate), $K=16$	16	2.13M	70.4 $\pm$ 25.2
RBF-GNN (multivariate), $K=27$	27	2.34M	19.0 $\pm$ 32.8

Table 2: FAUST shape correspondence. Accuracy (%) is reported as mean $\pm$ std over 3 seeds; the published SplineCNN result is from Fey et al. (2018). **best**, **second**, and **third** mark the three highest accuracy means and the three smallest parameter counts.

Small multivariate rational bases give the best and most stable results with substantially fewer parameters than SplineCNN. Their degradation at larger basis sizes points to fragile PCA initialization rather than insufficient capacity.

Event-based object recognition on N-Caltech101. We evaluate multiclass object recognition on N-Caltech101 (Orchard et al., 2015) with the seven-layer AEGNN recognition network (Schaefer et al., 2022). Each event is a graph node with space-time pseudo-coordinates; we replace only the SplineConv layers and report test accuracy over three random seeds.

Method	K	Aggr.	Params ↓	Final ↑
<i>Spline baselines</i>				
SplineCNN $k=2$	8	mean	77.68k	47.35 $\pm$ 0.15
SplineCNN $k=2$	8	max	77.68k	56.00 $\pm$ 0.92
<i>PointNet learned-kernel control</i>				
PointNet	—	max	55.68k	54.99 $\pm$ 0.84
<i>RBF-GNN with a multivariate basis</i>				
RBF-GNN (multivariate)	8	mean	91.57k	55.05 $\pm$ 0.58
RBF-GNN (multivariate)	8	max	91.57k	57.55 $\pm$ 0.42

Table 3: N-Caltech101 event-based object recognition with AEGNN. Models use training augmentation and validation-based convergence; test accuracy (%) is mean $\pm$ std over 3 seeds. **best**, **second**, and **third** mark the three highest means and the three smallest parameter counts.

RBF-GNN performs best with max aggregation and also improves substantially over SplineCNN under matched mean aggregation. Its advantage under both aggregation rules shows that the gain is not explained only by the neighborhood reducer.

Event-based car recognition on N-Cars. We evaluate binary car-versus-background recognition on the real-world N-Cars dataset (Sironi et al., 2018) with the same AEGNN architecture and replacement protocol (Schaefer et al., 2022). We report test accuracy after training for 30 epochs, averaged over three random seeds.

Method	K	Aggr.	Params ↓	Final ↑
<i>Spline baselines</i>				
SplineCNN $k=2$	8	mean	26.99k	87.83 ± 0.25
SplineCNN $k=2$	8	max	26.99k	89.81 ± 0.27
<i>PointNet learned-kernel control</i>				
PointNet	–	max	4.99k	87.17 ± 0.24
<i>RBF-GNN with a multivariate basis</i>				
RBF-GNN (multivariate)	4	mean	21.10k	90.24 ± 0.39
RBF-GNN (multivariate)	8	mean	40.88k	90.49 ± 0.05
RBF-GNN (multivariate)	8	max	40.88k	91.00 ± 0.43

Table 4: N-Cars event-based car recognition with AEGNN. Test accuracy (%) is mean $\pm$ std over 3 seeds; **best**, **second**, and **third** mark the three highest means and the three smallest parameter counts.

RBF-GNN is strongest under both aggregation rules, and its mean-aggregation variant outperforms the max-aggregation SplineCNN. The smaller RBF-GNN configuration also improves accuracy while using fewer parameters than SplineCNN.

Superpixel graph classification on MNIST. We evaluate graph classification on MNIST superpixels following the SplineCNN protocol (Fey et al., 2018). The network uses two continuous-kernel convolution layers with 2-D Cartesian pseudo-coordinates, and we report test accuracy over three random seeds.

Method	K	Params ↓	Final ↑
<i>Spline baselines</i>			
SplineCNN $k=5$	25	88.36k	97.41 ± 0.13
SplineCNN $k=3$	9	55.08k	96.49 ± 0.12
SplineCNN $k=2$	4	44.68k	94.83 ± 0.11
<i>MLP learned-basis control</i>			
MLP basis, $K=4$	4	45.59k	97.23 ± 0.11
<i>RBF-GNN with a multivariate basis</i>			
RBF-GNN (multivariate), PCA+vp, $K=4$	4	45.26k	97.79 ± 0.08
RBF-GNN (multivariate), PCA+vp, $K=9$	9	56.38k	98.00 ± 0.16
RBF-GNN (multivariate), hat init, $k=3$	9	56.38k	97.87 ± 0.06

Table 5: MNIST superpixel graph classification. Test accuracy (%) is mean $\pm$ std over 3 seeds; **best**, **second**, and **third** mark the three highest means and the three smallest parameter counts.

RBF-GNN gives the three strongest results and improves over SplineCNN at both matched and reduced basis sizes. Its advantage over the equal-width MLP control supports the rational parameterization itself.

Superpixel graph classification on CIFAR-10 with 5-D bilateral coordinates. We evaluate CIFAR-10 superpixel graph classification from GNNBenchmarkDataset (Dwivedi et al., 2023). The bilateral pseudo-coordinate concatenates relative 2-D centroid position and RGB features into a 5-D kernel, and we report test accuracy over three random seeds.

Method	K	Params ↓	Final ↑
<i>Spline baselines</i>			
SplineCNN $k=3$	243	557.42k	58.73 ± 0.26
SplineCNN $k=2$	32	105.03k	57.55 ± 0.34
<i>MLP learned-basis control</i>			
MLP basis, $K=8$	8	55.39k	59.67 ± 0.32
MLP basis, $K=32$	32	109.96k	59.02 ± 0.10
<i>RBF-GNN with a multivariate basis</i>			
RBF-GNN (multivariate), $K=4$	4	46.45k	57.58 ± 0.23
RBF-GNN (multivariate), $K=8$	8	56.47k	58.97 ± 0.16
RBF-GNN (multivariate), $K=32$	32	116.62k	59.87 ± 0.40

Table 6: CIFAR-10 superpixel graph classification with 5-D bilateral pseudo-coordinates. Test accuracy (%) is mean $\pm$ std over 3 seeds; **best**, **second**, and **third** mark the three highest means and the three smallest parameter counts.

RBF-GNN attains the best accuracy while using far fewer parameters than the wider spline grid. The lower-width sweep remains competitive, although the MLP control is strongest at the smallest shared width.

Part segmentation on ShapeNet-Part with 6-D coordinates. We evaluate point-cloud part segmentation on ShapeNet-Part (Yi et al., 2016) using a fixed subset of 4,000 training and 1,333 test shapes. Relative positions and surface normals define 6-D pseudo-coordinates on a 16-nearest-neighbor graph, and we report instance-average mIoU over three random seeds.

Method	K	Params ↓	Final mIoU ↑
<i>Spline baselines</i>			
SplineCNN $k=3$	729	1.60M	65.12 ± 0.41
SplineCNN $k=2$	64	177.91k	60.02 ± 0.23
<i>MLP learned-basis control</i>			
MLP basis, $K=8$	8	60.75k	62.80 ± 0.45
MLP basis, $K=32$	32	116.88k	63.81 ± 0.46
<i>RBF-GNN with a multivariate basis</i>			
RBF-GNN (multivariate), $K=8$	8	64.87k	61.88 ± 0.38
RBF-GNN (multivariate), $K=32$	32	137.43k	64.86 ± 0.18

Table 7: ShapeNet-Part segmentation with 6-D position-normal pseudo-coordinates. Instance-average mIoU (%) is mean $\pm$ std over 3 seeds; **best**, **second**, and **third** mark the three highest means and the three smallest parameter counts.

RBF-GNN nearly matches the large spline grid with an order-of-magnitude smaller model and outperforms the equal-width MLP control. At the narrowest width the MLP is stronger, so this benchmark supports efficient scaling rather than universal gains at every capacity.

Keypoint matching on WILLOW-ObjectClass. We evaluate semantic keypoint matching on WILLOW-ObjectClass with the DGMC pipeline (Fey et al., 2020), without PascalVOC pretraining. We report final matching accuracy over ten runs and include an initialization ablation for the product rational basis.

Method	Final $\uparrow$
<i>Spline baselines</i>	
SplineCNN	96.80 $\pm$ 1.20
<i>RBF-GNN with a product basis</i>	
RBF-GNN (product), hat init	97.28 $\pm$ 1.01
RBF-GNN , Gaussian init	97.72
RBF-GNN , Chebyshev+vp init	97.07
RBF-GNN , random+vp init	92.13
RBF-GNN , constant init	10.00
<i>RBF-GNN with a multivariate basis</i>	
RBF-GNN (multivariate), $k=5$	97.55 $\pm$ 0.63

Table 8: WILLOW-ObjectClass keypoint matching. Final accuracy (%) is shown; the backbone comparison reports mean $\pm$ std over 10 runs. **best**, **second**, and **third** mark the three highest means.

The multivariate model gives the best mean accuracy, although the backbone differences are not statistically significant. Initialization strongly affects optimization, and a constant basis collapses to chance performance.

Keypoint matching on PascalPF. We evaluate semantic keypoint matching on synthetic and PascalPF image pairs following DGM (Fey et al., 2020). We report final matching accuracy over three random seeds and compare the initialization strategies used for the product rational basis.

Method	Final $\uparrow$
<i>Spline baselines</i>	
SplineCNN, $k=5$	95.27 $\pm$ 0.15
<i>RBF-GNN with a product basis</i>	
RBF-GNN (product), $k=3$, hat init	95.73 $\pm$ 0.06
RBF-GNN , Gaussian init	95.90
RBF-GNN , random init	94.93
RBF-GNN , constant init (two variants)	78 / 66

Table 9: PascalPF keypoint matching. Final accuracy (%) is reported as mean $\pm$ std over 3 seeds where available. **best**, **second**, and **third** mark the three highest means.

The rational basis has higher mean accuracy than the spline baseline, although the difference remains within seed variability. The poor constant initialization again shows that a non-degenerate starting basis is essential.

AI USE STATEMENT

[TODO — required by ICLR 2027. Disclose which generative AI tools were used and for what.]

ETHICS STATEMENT

[TODO — recommended.]

REPRODUCIBILITY STATEMENT

[TODO — recommended. Point at the released code, the named configurations in `slurm/configs.sh`, the per-run training logs under `results/logs/`, and the fixed seeds used for each table.]

REFERENCES

- Randall Balestriero, Romain Cosentino, Hervé Glotin, and Richard Baraniuk. Spline filters for end-to-end deep learning. In *International Conference on Machine Learning (ICML)*, pp. 364–373, 2018.
- Nicolas Boullé, Yuji Nakatsukasa, and Alex Townsend. Rational neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pp. 14243–14253, 2020.
- Vijay Prakash Dwivedi, Chaitanya K. Joshi, Thomas Laurent, Yoshua Bengio, and Xavier Bresson. Benchmarking graph neural networks. *Journal of Machine Learning Research*, 24(43):1–48, 2023.
- Matthias Fey, Jan Eric Lenssen, Frank Weichert, and Heinrich Müller. SplineCNN: Fast geometric deep learning with continuous B-spline kernels. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 869–877, 2018.
- Matthias Fey, Jan Eric Lenssen, Christopher Morris, Jonathan Masci, and Nils M. Kriege. Deep graph matching consensus. In *International Conference on Learning Representations (ICLR)*, 2020.
- Justin Gilmer, Samuel S. Schoenholz, Patrick F. Riley, Oriol Vinyals, and George E. Dahl. Neural message passing for quantum chemistry. In *International Conference on Machine Learning (ICML)*, pp. 1263–1272, 2017.
- Ziming Liu, Yixuan Wang, Sachin Vaidya, Fabian Ruehle, James Halverson, Marin Soljačić, Thomas Y. Hou, and Max Tegmark. KAN: Kolmogorov–arnold networks. In *International Conference on Learning Representations (ICLR)*, 2025.
- Liheng Ma, Soumyasundar Pal, Yitian Zhang, Jiaming Zhou, Yingxue Zhang, and Mark Coates. CKGConv: General graph convolution with continuous kernels. In *International Conference on Machine Learning (ICML)*, pp. 33902–33924, 2024.
- Alejandro Molina, Patrick Schramowski, and Kristian Kersting. Padé activation units: End-to-end learning of flexible activation functions in deep networks. In *International Conference on Learning Representations (ICLR)*, 2020.
- Federico Monti, Davide Boscaini, Jonathan Masci, Emanuele Rodolà, Jan Svoboda, and Michael M. Bronstein. Geometric deep learning on graphs and manifolds using mixture model CNNs. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 5115–5124, 2017.
- Garrick Orchard, Ajinkya Jayawant, Gregory K. Cohen, and Nitish Thakor. Converting static image datasets to spiking neuromorphic datasets using saccades. *Frontiers in Neuroscience*, 9:437, 2015. doi: 10.3389/fnins.2015.00437.
- David W. Romero, Robert-Jan Bruintjes, Erik J. Bekkers, Jakub M. Tomczak, Mark Hoogendoorn, and Jan C. van Gemert. FlexConv: Continuous kernel convolutions with differentiable kernel sizes. In *International Conference on Learning Representations (ICLR)*, 2022a.
- David W. Romero, Anna Kuzina, Erik J. Bekkers, Jakub M. Tomczak, and Mark Hoogendoorn. CKConv: Continuous kernel convolution for sequential data. In *International Conference on Learning Representations (ICLR)*, 2022b.
- Anit Kumar Sahu, Rizal Fathony, Devin Willmott, and J. Zico Kolter. Multiplicative filter networks. In *International Conference on Learning Representations (ICLR)*, 2021.
- Simon Schaefer, Daniel Gehrig, and Davide Scaramuzza. AEGNN: Asynchronous event-based graph neural networks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 12371–12381, 2022.
- Kristof T. Schütt, Pieter-Jan Kindermans, Huziel E. Sauceda Felix, Stefan Chmiela, Alexandre Tkatchenko, and Klaus-Robert Müller. SchNet: A continuous-filter convolutional neural network for modeling quantum interactions. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017.

- Martin Simonovsky and Nikos Komodakis. Dynamic edge-conditioned filters in convolutional neural networks on graphs. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 3693–3702, 2017.
- Amos Sironi, Manuele Brambilla, Nicolas Bourdis, Xavier Lagorce, and Ryad Benosman. HATS: Histograms of averaged time surfaces for robust event-based object classification. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 10211–10220, 2018.
- Vincent Sitzmann, Julien N. P. Martel, Alexander W. Bergman, David B. Lindell, and Gordon Wetzstein. Implicit neural representations with periodic activation functions. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pp. 7462–7473, 2020.
- Matthew Tancik, Pratul P. Srinivasan, Ben Mildenhall, Sara Fridovich-Keil, Nithin Raghavan, Utkarsh Singhal, Ravi Ramamoorthi, Jonathan T. Barron, and Ren Ng. Fourier features let networks learn high frequency functions in low dimensional domains. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pp. 7537–7547, 2020.
- Hugues Thomas, Charles R. Qi, Jean-Emmanuel Deschaud, Beatriz Marcotegui, François Goulette, and Leonidas J. Guibas. KPConv: Flexible and deformable convolution for point clouds. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 6411–6420, 2019.
- Nitika Verma, Edmond Boyer, and Jakob Verbeek. FeaStNet: Feature-steered graph convolutions for 3D shape analysis. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 2598–2606, 2018.
- Shenlong Wang, Simon Suo, Wei-Chiu Ma, Andrei Pokrovsky, and Raquel Urtasun. Deep parametric continuous convolutional neural networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 2589–2597, 2018.
- Wenxuan Wu, Zhongang Qi, and Li Fuxin. PointConv: Deep convolutional networks on 3D point clouds. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 9621–9630, 2019.
- Mutian Xu, Runyu Ding, Hengshuang Zhao, and Xiaojuan Qi. PAConv: Position adaptive convolution with dynamic kernel assembling on point clouds. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 3173–3182, 2021.
- Yifan Xu, Tianqi Fan, Mingye Xu, Long Zeng, and Yu Qiao. SpiderCNN: Deep learning on point sets with parameterized convolutional filters. In *European Conference on Computer Vision (ECCV)*, pp. 87–102, 2018.
- Xingyi Yang and Xinchao Wang. Kolmogorov–Arnold transformer. In *International Conference on Learning Representations (ICLR)*, 2025.
- Li Yi, Vladimir G. Kim, Duygu Ceylan, I-Chao Shen, Mengyan Yan, Hao Su, Cewu Lu, Qixing Huang, and Leonidas J. Guibas. A scalable active framework for region annotation in 3D shape collections. *ACM Transactions on Graphics*, 35(6):210:1–210:12, 2016. doi: 10.1145/2980179.2980238.
- Hengshuang Zhao, Li Jiang, Jiaya Jia, Philip H. S. Torr, and Vladlen Koltun. Point transformer. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 16259–16268, 2021.